

Software can cause the system to hang indefinitely, in case of an infinite loop, buffer overflow or deadlocks. In a small embedded device, it is easy to find the exact root cause of the bug, but not so in a complex embedded system. However, by using a watchdog, we can ensure that the system will not hang indefinitely.

Hence, the system software in any situation should not hang infinitely. A general solution, in case it does hang, is to reset the system, and this is where watchdogs in embedded systems come in handy.

Watchdog timer based system design

The software needs to kick the watchdog constantly. In some implementations, a sequence of bytes is needed to be written in the watchdog register to kick the watchdog. This reduces the chance of an errant code that might accidentally kick the watchdog.

After WDT overflows, it will assert the processor reset line. Some processors and controllers can generate an interrupt before resetting the device, which is like an early warning for an upcoming watchdog reset. We can save useful information like status register in a non-volatile memory by reading this information after recovery. From reset logs, we can debug the root cause of the reset.

A watchdog can also be used to wake up the device from sleep or idle mode. In sleep mode, watchdog timeout will not reset the system, but just cause it to wake up.

Simply enabling WDT and kicking it regularly is not enough to ensure system reliability. To get optimum benefit, implementation of the watchdog is a must for robust design.

Watchdog time-out period

For selecting watchdog time-out period, we must have a proper understanding of the software loop latency. An unusual number of interrupts may happen during a single scanning of a loop, and the extra time spent

in the interrupt service routine (ISR) will increase the main loop latency. A software delay routine will also increase loop latency. The design with delays in various places in the code has control of the watchdog, which can prove to be problematic.

For some time, critical application and recovery time from the watchdog reset is very important. In such a system, time-out period needs to be very precise. After watchdog reset, the system must boot-up as fast as possible. For example, in case of a pacemaker machine, the system must boot-up almost within a heartbeat. The initialisation after a watchdog reset should be much shorter than power-on initialisation.

Very short time-out periods may lead to the system resetting unnecessarily. If the system is not time-critical, it is better to choose time-out in seconds.

Implementation of watchdog timer for single-thread software design

The traditional approach for a single-thread design is to kick WDT at the end of the main loop.

In a single-thread design, we can use state-machine-like architecture as shown in the code snippet below. Increment the state variable value at three different sections of the code, which will definitely iterate once in a one-loop scan. At the end of the main loop, check the state value;

if it is three, it means that the code execution is done in proper sequence. Then, kick the watchdog and clear the state flag. If the state value is not three, it means there is some fault in the execution of the code. In this case, do not kick the watchdog, else the system will reset after watchdog time-out.

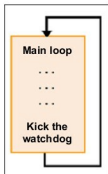


Fig. 3: Traditional watchdog kicking inside the main loop

```

-----CODE-----
main ()
{
    for( ; ; )
    {
        if(State == 0) State = 0x01;
        . . .
        . . .
        if(State == 1) State = 0x02;
        . . .
        . . .
        if(State == 2) State = 0x03;
        . . .
        . . .
        If (State == 0x03)
        {
            Kick the watchdog
            State = 0;
        }
    }
}
-----CODE-----
  
```

On some microcontrollers, the built-in watchdog has a maximum time-out of the order of a few hundred milliseconds. But, if the main loop scan time is higher than the maximum allowed watchdog time-out, we need to multiply that in the software.

For example, main loop latency of 500ms and maximum allowed watchdog time-out period of 100ms (which means that the watchdog must kick before 100ms) is not possible from the main loop. In this case, we can configure the processor's internal timer to 50ms free-running and define flag state at the end of the main loop set and state it as Alive.

```

-----CODE-----
main ()
{
    for ( ; ; )
    {
        . . .
        . . .
        State = ALIVE;
    }
}
-----CODE-----
  
```

In every 50ms ISR increment count, check state flag. Only kick the watchdog if state is not Unknown.